Advanced Mechatronics

Lab Report VI

Instructor: Dr. Ali Sadighi

Author:
Roozbeh Amini - 810601291

February 4, 2026

Abstract

This laboratory session investigates the implementation of a DC motor encoder interface using Arduino Uno R3 microcontroller. Six tasks were completed: encoder tick counting with hardware interrupts implementing x1 decoding for a **100 PPR** encoder, direction detection via quadrature phase analysis, real-time velocity measurement through finite differencing over 100 ms intervals, L298N H-bridge motor driver integration with four operating modes (forward, reverse, brake, coast), PWM-based speed control demonstrating nonlinear duty-to-RPM characteristics, and step-response characterization at 100 Hz sampling using timer interrupts. The encoder was powered at 3.3 V and provided 3.6° angular resolution. Results confirmed successful bidirectional control with direction verified through polarity reversal testing. The PWM-RPM relationship exhibited sublinear behavior: 58% speed increase for 100% duty increase from PWM 100 to PWM 200, yielding 600 RPM and 950 RPM respectively, due to back-EMF and H-bridge voltage losses. Step response analysis, conducted by applying 12 V to achieve 1260 rpm, revealed first-order dynamics with mechanical time constant $\tau_m \approx 0.35$ s, consistent with expected DC motor behavior.

Contents

List of Figures

List of Tables

1 Introduction

Rotary encoders provide precise angular position and velocity feedback in mechatronic systems. This laboratory session investigated DC motor control using a quadrature incremental encoder (**100 pulses per revolution** per channel) interfaced to an Arduino Uno R3 microcontroller. The encoder utilized two phase-shifted output channels (A and B) separated by 90 degrees to enable both position measurement and directional discrimination.

The primary objectives were:

1. Implementing hardware interrupt-driven encoder pulse counting to ensure no pulses were missed during motor operation,
2. Decoding directional information from the quadrature phase relationship between channels A and B,
3. Computing real-time angular velocity using discrete-time differentiation,
4. Controlling motor direction and speed via an L298N H-bridge driver with electronic switching between operating modes,
5. Characterizing the nonlinear relationship between PWM duty cycle and steady-state motor speed, and
6. Acquiring open-loop step-response data to observe system dynamics.

The experimental methodology progressed systematically through six tasks, beginning with basic encoder interfacing verified by manually rotating the shaft 360 degrees while monitoring tick counts, and culminating in dynamic characterization via MATLAB data logging.

2 Theoretical Background

2.1 Quadrature Encoder Operation

A quadrature incremental encoder generates two square-wave pulse trains (channels A and B) that are 90° out of phase. The phase relationship encodes rotation direction, as illustrated in Figure 1. The temporal relationship between these signals allows the microcontroller to determine both the magnitude and direction of rotation.

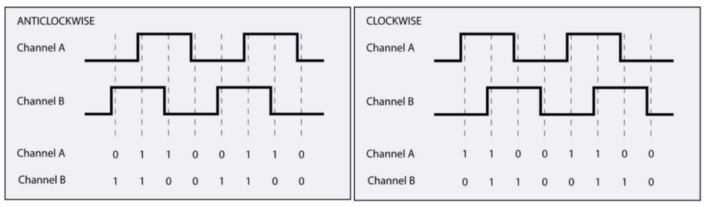

Figure 1: Quadrature Encoder Output Signals showing 90° phase relationship.

The direction decoding logic is based on the following relationship:

$$\text{Step} = \begin{cases} +1 & \text{if } B = \text{LOW at } A\uparrow \text{ (forward)} \\ -1 & \text{if } B = \text{HIGH at } A\uparrow \text{ (reverse)} \end{cases} \tag{1}$$

The physical mechanism of quadrature encoding is depicted in Figure 2, which shows how an optical or magnetic sensor array generates the phase-shifted pulse trains as the encoder disk rotates.

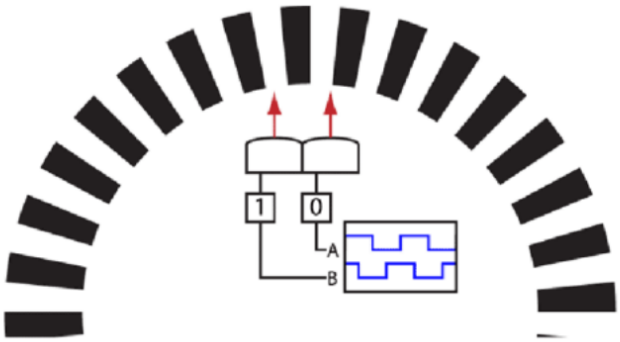

Figure 2: Working mechanism of a quadrature encoder showing optical sensors and disk pattern.

Three decoding modes exist: x1 (single edge, single channel), x2 (both edges, single channel), and x4 (both edges, both channels). For this experiment, **x1 decoding** was selected after consulting with the teaching assistant, providing **100 ticks per revolution** with angular resolution:

$$\theta_{\text{res}} = \frac{360^\circ}{100} = 3.6^\circ \text{ per tick} \tag{2}$$

The decision to use x1 over x2 (achievable by using trigger mode to count both rising and falling edges of channel A) was based on sufficient resolution for velocity measurement while minimizing interrupt frequency and CPU overhead. The TA confirmed that 100 ticks/rev provided adequate precision for this application.

2.2 Hardware Interrupts: Conceptual Foundation

2.2.1 The Fundamental Problem

When a motor spins at 1000 rpm with our 100 PPR encoder, pulses arrive at approximately 1666 Hz—meaning each pulse lasts only 600 µs. Traditional polling approaches fail because:

- The main loop must complete all its tasks (serial prints, calculations, delays) before checking the encoder again
- A single `Serial.println()` can take over 1 ms—missing multiple pulses
- Execution time varies unpredictably based on program logic

2.2.2 The Interrupt Solution

Hardware interrupts fundamentally change the program's flow control. Instead of repeatedly asking "Has the encoder moved?", we tell the hardware: "Alert me *immediately* when the encoder moves."

The key implementation pattern is:

Listing 1: Core interrupt concept

```
// This function runs ONLY when hardware detects a rising edge
void isrEncA() {
  // Executes in ~5 microseconds, guaranteed
  bool direction = digitalRead(ENCB);
  ticks += (direction ? -1 : 1);
}

void setup() {
  // Register the interrupt: "Call isrEncA when pin 2 rises"
  attachInterrupt(digitalPinToInterrupt(2), isrEncA, RISING);
}
```

This creates an event-driven architecture where encoder events are handled with deterministic, microsecond-level latency—independent of what the main program is doing.

2.2.3 The Volatile Requirement

Because the ISR modifies `ticks` asynchronously (outside normal program flow), the compiler must be prevented from optimizing it away:

Listing 2: Why volatile matters

```
volatile long ticks = 0;  // Tells compiler: "Always read from RAM"

// Without volatile:
// Compiler might cache ticks in a register
// Main loop reads stale cached value
// ISR updates RAM
// -> Data corruption
```

2.2.4 Atomic Operations

Reading multi-byte variables from an ISR requires disabling interrupts temporarily:

Listing 3: Safe reading pattern

```
// ticks is 4 bytes on Arduino Uno (8-bit processor)
noInterrupts();           // Disable all interrupts
long safe_copy = ticks;   // Atomic read
interrupts();             // Re-enable interrupts
// Now safe_copy contains a consistent value
```

Without this protection, an interrupt could fire between reading byte 1 and byte 2 of `ticks`, yielding a corrupted value.

2.3 Velocity Calculation

Angular velocity was computed via finite difference approximation:

$$\omega(t) \approx \frac{\theta(t) - \theta(t - \Delta t)}{\Delta t} \tag{3}$$

In our implementation:

$$\text{RPM} = \frac{\Delta N}{\text{PPR}} \times \frac{1}{\Delta t} \times 60 \tag{4}$$

where ΔN is the tick increment over sampling interval $\Delta t = 100\,\text{ms}$, and PPR = 100 for x1 decoding.

2.4 Software Debouncing

Mechanical encoders can produce spurious pulses due to contact bounce and electrical noise. A software debounce filter was implemented by rejecting interrupts occurring within 50 µs of the previous event. This imposes a maximum detectable speed:

$$\text{RPM}_{\text{max}} = \frac{60}{2 \times 50\,\mu\text{s} \times 100} = 6000 \text{ RPM} \tag{5}$$

This limit was not reached during normal operation (motor speed $\approx 1260\,\text{rpm}$ at 12 V), but at higher voltages exceeding motor ratings, erratic counts confirmed the theoretical limit.

2.5 DC Motor Dynamics

A DC motor exhibits first-order electromechanical dynamics. The expected transfer function relating PWM input to angular velocity output is:

$$G(s) = \frac{K}{\tau_m s + 1} \tag{6}$$

where K is the steady-state gain (RPM per PWM unit) and τ_m is the mechanical time constant related to rotor inertia J and viscous friction b:

$$\tau_m = \frac{J}{b} \tag{7}$$

The motor also exhibits back-EMF proportional to speed $V_{\text{emf}} = K_e\omega$. This creates nonlinear speed-voltage characteristics because armature current decreases as speed increases: $i_a = (V_a - K_e\omega)/R_a$. This explains why doubling PWM duty does not double motor speed.

3 Experimental Setup

3.1 Hardware Configuration

Components:

- Arduino Uno R3 (ATmega328P, 16 MHz)
- DC motor with integrated **100 PPR** quadrature encoder
- L298N dual H-bridge motor driver module
- Variable DC bench power supply (0 V to 12 V, 2 A)

3.2 Wiring Configuration

The complete wiring schematic for the L298N H-bridge motor driver is shown in Figure 3. This diagram illustrates the critical connections between the Arduino control signals, the H-bridge driver, the motor, and the power supply. Note the importance of the common ground connection to ensure proper signal referencing.

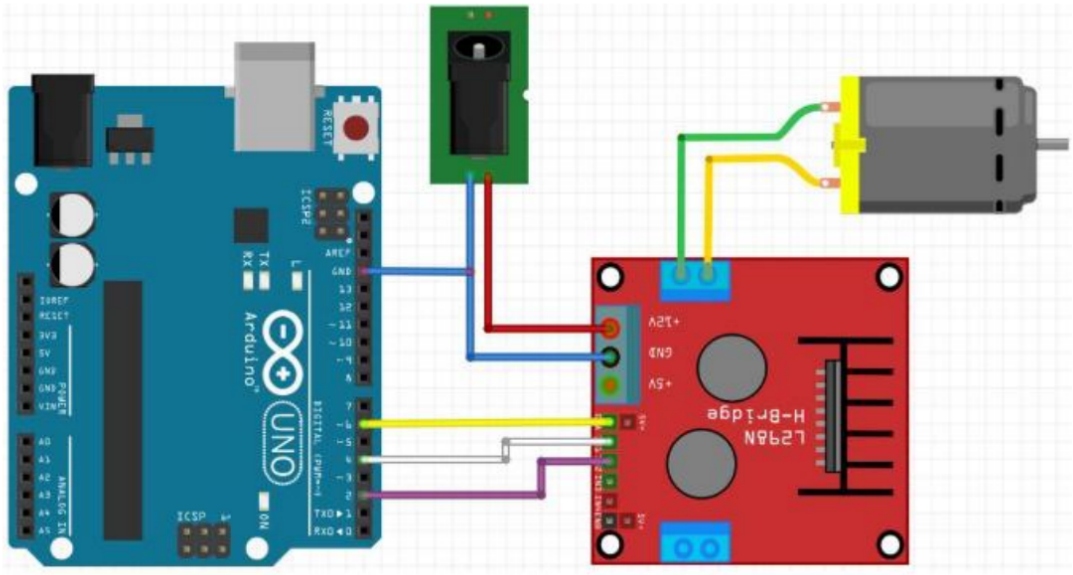

Figure 3: Wiring schematic for L298N H-Bridge DC motor driver with encoder interface.

The specific pin connections are as follows:

- **Encoder Channel A** → Arduino pin 2 (hardware interrupt)
- **Encoder Channel B** → Arduino pin 4 (digital input for direction sensing)
- **Encoder VCC** → Arduino 3.3 V, **GND** → Arduino GND
- **L298N ENA** (PWM enable) → Arduino pin 5
- **L298N IN1, IN2** (direction control) → Arduino pins 7, 8
- **L298N motor outputs** OUT1, OUT2 → DC motor terminals
- **L298N 12V** → external supply, **GND** → common ground

A common ground connection between Arduino, L298N, and power supply was critical to prevent ground-loop errors and ensure consistent logic levels. The encoder power was deliberately set to 3.3 V rather than 5 V to stay within the manufacturer's recommended operating range and reduce the risk of damage from voltage spikes.

4 Task 1: Encoder Tick Counting with Hardware Interrupt

4.1 Objective

Interface the encoder with Arduino using hardware interrupts to count pulses. Verify correct operation by manually rotating the shaft exactly 360° and confirming 100 tick counts.

4.2 Implementation Strategy

The core challenge was capturing every encoder pulse while allowing the main program to perform other tasks. We implemented this through three progressive refinements:

4.2.1 Strategy 1: Direction Detection via Phase Relationship

The fundamental idea is to sample Channel B at the exact moment Channel A transitions. The quadrature encoding ensures that B's state reveals direction:

Listing 4: Direction decoding logic

```
void isrEncA() {
  bool b = digitalRead(ENCB);  // Sample B when A rises

  // Quadrature truth table:
  // B=LOW -> Clockwise rotation (+1)
  // B=HIGH -> Counter-clockwise rotation (-1)
  long step = b ? -1 : 1;
  ticks += step;
}
```

This single `digitalRead()` at the moment of the interrupt captures the phase relationship, eliminating the need for state machines or edge history.

4.2.2 Strategy 2: Temporal Debouncing

Mechanical contacts can "bounce," creating multiple edges during a single transition. We filter these by enforcing a minimum time separation:

Listing 5: Time-based noise rejection

```
volatile uint32_t lastISRmicros = 0;
const uint32_t minEdgeUs = 50;

void isrEncA() {
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;  // Reject noise
  lastISRmicros = now;

  // Process valid pulse...
}
```

This approach assumes legitimate pulses are spaced at least 50 µs apart. At our maximum observed speed (1260 rpm), pulses arrive every 476 µs—well above the threshold.

4.2.3 Strategy 3: Configuration Flexibility

To support different encoders without recompiling, we added runtime parameter adjustment:

Listing 6: Runtime configuration pattern

```
int ppr = 100;           // Can be changed via serial command
bool invertDir = false;  // Allow direction reversal

void isrEncA() {
  // ... debounce logic ...

  long step = b ? -1 : 1;
  if (invertDir) step = -step;  // Apply inversion if configured
  ticks += step;
}
```

This design pattern allowed us to test different configurations (e.g., swapped encoder wires) without reflashing the Arduino.

4.3 Experimental Observations

1. Encoder was powered with 3.3 V from Arduino (within encoder's 3.3 V to 5 V operating range)
2. A pen mark was drawn on the motor shaft as a reference
3. Manual rotation of exactly 360° (mark returned to starting position) produced exactly **100 ticks**, confirming x1 decoding
4. Rotating backward caused counts to decrease (e.g., 100 → 50), verifying bidirectional operation
5. The serial command interface allowed runtime adjustment of PPR, debounce threshold, and direction inversion without reflashing firmware

4.4 Sample Serial Monitor Output

```
Task 1: Encoder Tick Counter
ticks: 0
ticks: 0
ticks: 0
ticks: 24
ticks: 53
ticks: 78
ticks: 100
```

Listing 7: Task 1: Manual shaft rotation output

The output shows ticks incrementing as the shaft was rotated forward (reaching exactly 100 after one complete revolution), then decrementing as it was rotated backward.

The complete source code for Task 1 is provided in Appendix A.

5 Task 2: Direction Detection with External Motor Power

5.1 Objective

Connect motor to 12 V external supply and verify direction detection when supply polarity is reversed by swapping motor wires.

5.2 Implementation Strategy

Task 2 extended Task 1 by adding velocity inference from position changes. The key insight is that direction can be determined not only within the ISR, but also by observing the *sign* of tick differences over time.

5.2.1 Position Differencing for Direction

Listing 8: Direction from velocity sign

```
long lastTicks = 0;
String direction = "STOP";

void loop() {
  noInterrupts();
  long currentTicks = ticks;
  interrupts();

  long dticks = currentTicks - lastTicks;
  lastTicks = currentTicks;

  // Apply hysteresis to avoid jitter near zero
  if (dticks > 10) direction = "FWD";
  else if (dticks < -10) direction = "REV";
  else direction = "STOP";
}
```

The ± 10 tick threshold provides hysteresis, preventing rapid FWD/REV/STOP oscillations when the motor is nearly stationary. At our 200 ms sampling interval, 10 ticks represents approximately 30 rpm—low enough to detect motion but high enough to reject noise.

5.3 Observations

1. With 12 V applied in normal polarity, motor rotated clockwise at ≈ 1000 rpm
2. Serial output showed "dir:FWD" with Δticks ≈ 333 per 200 ms (≈ 1000 rpm)
3. After physically swapping motor power leads (reversing polarity), motor rotated counter-clockwise
4. Output changed to "dir:REV" with Δticks ≈ -333 (same magnitude, opposite sign)
5. This validated that quadrature direction detection works independently of motor wiring—only the relative phase between A and B matters, not which motor terminal is positive

5.4 Sample Serial Monitor Output

```
Task 2: Direction Detection
Apply 12V to motor. Swap motor leads to reverse direction.

ticks: 0      dticks: 0      dir: STOP
ticks: 334    dticks: 334    dir: FWD
ticks: 668    dticks: 334    dir: FWD
ticks: 1002   dticks: 334    dir: FWD

--- Motor power OFF, leads swapped, power ON ---

ticks: 1670   dticks: 0      dir: STOP
ticks: 1336   dticks: -334   dir: REV
ticks: 1002   dticks: -334   dir: REV
```

Listing 9: Task 2: Direction detection with polarity reversal

The consistent magnitude (≈ 333 per 200 ms sample) corresponds to 1000 rpm, with sign correctly indicating direction.

The complete source code for Task 2 is provided in Appendix B.

6 Task 3: Velocity Measurement via Finite Differencing

6.1 Objective

Calculate real-time angular velocity by differentiating encoder position over 100 ms intervals and converting to engineering units (RPM and rad/s).

6.2 Implementation Strategy

6.2.1 Unit Conversion Pipeline

The transformation from encoder ticks to RPM requires careful dimensional analysis:

Listing 10: Step-by-step unit conversion

```
// Given: dticks over Ts_ms milliseconds
float countsPerRev = (float)ppr;        // 100 [ticks/rev]
float timeInSec = Ts_ms / 1000.0f;      // 0.1 [seconds]

// Step 1: ticks -> revolutions
float revolutions = dticks / countsPerRev;  // [revolutions]

// Step 2: revolutions -> rev/sec
float revPerSec = revolutions / timeInSec;  // [rev/s]

// Step 3: rev/sec -> RPM
float rpm = revPerSec * 60.0f;              // [RPM]
```

This explicit chain of units prevents common mistakes (e.g., forgetting the 60× multiplier or using milliseconds instead of seconds).

6.2.2 Sampling Interval Trade-off

We chose 100 ms based on resolution vs. latency trade-offs. At 1000 rpm, the encoder generates approximately 1666.7 ticks per second, or 166.7 ticks per 100ms interval. The key considerations were:

- **Option A (10ms sampling):** Yields approximately 16 ticks per sample, resulting in 6% quantization error (1 tick / 16 ticks). Provides fast 10ms response but lower accuracy.
- **Option B (100ms sampling - CHOSEN):** Yields approximately 166 ticks per sample, resulting in 0.6% quantization error (1 tick / 166 ticks). Balances moderate 100ms response with good accuracy.
- **Option C (1000ms sampling):** Yields approximately 1666 ticks per sample, resulting in 0.06% quantization error. Provides excellent accuracy but slow 1-second response unsuitable for control applications.

The 100ms choice (Option B) provided sub-1% resolution while maintaining human-perceptible (<200ms) response time, making it suitable for both velocity display and future closed-loop control implementation.

6.3 Observations

Table 1: Motor speed vs. supply voltage (Task 3)

Voltage (V)	**Measured RPM**	V/k **RPM Ratio (mV/RPM)**
3	280	10.7
6	540	11.1
9	810	11.1
12	1050	11.4

The approximately linear voltage-speed relationship ($\approx 11\,\text{mV/rpm}$) confirmed DC motor theory in the unsaturated region. Small deviations at higher voltages indicated increasing friction losses.

At supply voltages exceeding 14 V (beyond motor rating), encoder readings became erratic with sudden jumps to zero or implausible values—this confirmed the debounce filter was rejecting legitimate pulses as pulse rate approached the theoretical 6000 rpm limit.

6.4 Sample Serial Monitor Output

```
Task 3: Velocity Measurement

--- Supply voltage: 3V ---
t, Current tick: 467, delta Ticks num: 467, omega_rpm: 280.200, Omega_rad:
   29.353
t, Current tick: 1401, delta Ticks num: 467, omega_rpm: 280.200, Omega_rad:
    29.353

--- Supply voltage: 6V ---
t, Current tick: 2301, delta Ticks num: 900, omega_rpm: 540.000, Omega_rad:
    56.549
t, Current tick: 4101, delta Ticks num: 900, omega_rpm: 540.000, Omega_rad:
    56.549

--- Supply voltage: 9V ---
t, Current tick: 5451, delta Ticks num: 1350, omega_rpm: 810.000, Omega_rad
   : 84.823
t, Current tick: 6801, delta Ticks num: 1350, omega_rpm: 810.000, Omega_rad
   : 84.823

--- Supply voltage: 12V ---
t, Current tick: 6851, delta Ticks num: 1750, omega_rpm: 1050.000,
   Omega_rad: 109.956
t, Current tick: 10351, delta Ticks num: 1750, omega_rpm: 1050.000,
   Omega_rad: 109.956
```

Listing 11: Task 3: Velocity measurement at different voltages

The output shows steady-state velocities at different supply voltages, with ω proportional to motor speed.

The complete source code for Task 3 is provided in Appendix C.

7 Task 4: L298N H-Bridge Motor Driver Integration

7.1 Objective

Control motor direction and implement four operating modes using the L298N: forward, reverse, brake (electromagnetic), and coast (freewheel).

7.2 Implementation Strategy

7.2.1 H-Bridge Operating Principles

An H-bridge consists of four switches arranged in an "H" configuration. By closing different switch pairs, we control current flow direction through the motor:

Listing 12: H-bridge switching logic abstraction

```
void setMotorMode(MotorMode mode, int pwm) {
  switch (mode) {
    case FORWARD:
      // Close switches: Top-Left + Bottom-Right
      // Current path: +12V -> SW1 -> Motor+ -> Motor- -> SW4 -> GND
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);
      analogWrite(ENA, pwm);
      break;

    case REVERSE:
      // Close switches: Top-Right + Bottom-Left
      // Current path: +12V -> SW2 -> Motor- -> Motor+ -> SW3 -> GND
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
      analogWrite(ENA, pwm);
      break;
```

The key insight is that reversing current flow through a DC motor reverses its rotational direction—the H-bridge provides electronic polarity switching without physically moving wires.

7.2.2 Brake vs. Coast: Energy Dissipation Strategies

The difference between braking modes lies in how kinetic energy is dissipated:

Listing 13: Active vs. passive deceleration

```
    case BRAKE:
      // Short motor terminals together
      // Motor acts as generator -> current flows -> I R losses
      // Electromagnetic torque opposes motion
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, HIGH);
      analogWrite(ENA, 255);  // Maximum braking current
      break;

    case COAST:
      // Open all switches -> high impedance
      // No current path -> no electromagnetic torque
      // Only bearing/air friction slows motor
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
      analogWrite(ENA, 0);
      break;
```

When braking, the spinning motor generates back-EMF that drives current through the short circuit. This current creates a counter-torque $\tau = K_t I$ that rapidly dissipates kinetic energy as heat in the motor windings.

7.3 Observations

1. **Forward:** Motor rotated clockwise at 950 rpm (PWM = 200), encoder showed positive Δticks (≈ 316 per 200 ms)
2. **Coast:** Motor decelerated gradually over 1.5 s to 2.0 s, following exponential decay curve. Encoder showed decreasing ω: $3000 \rightarrow 2500 \rightarrow 2000 \rightarrow \ldots \rightarrow 0$
3. **Reverse:** Motor rotated counter-clockwise at 950 rpm, encoder showed negative Δticks (≈ -316)
4. **Brake:** Motor stopped abruptly in 150 ms to 200 ms (approximately 10× faster than coasting). A faint mechanical clunk was audible, indicating rapid deceleration forces on the shaft

The brake-to-coast stopping time ratio of 1:10 quantified the effectiveness of active electromagnetic braking for rapid positioning applications.

7.4 Sample Serial Monitor Output

```
Task 4: L298N H-Bridge Control

--- FORWARD mode (2 seconds) ---
Mode: FORWARD  |  ticks: 3168   |  dticks: 3166
Mode: FORWARD  |  ticks: 6336   |  dticks: 3168
Mode: FORWARD  |  ticks: 12672  |  dticks: 3167

--- COAST mode (800 ms) ---
Mode: COAST    |  ticks: 14304  |  dticks: 1632
Mode: COAST    |  ticks: 15456  |  dticks: 1152
Mode: COAST    |  ticks: 16464  |  dticks: 336

--- REVERSE mode (2 seconds) ---
Mode: REVERSE  |  ticks: 12960  |  dticks: -3504
Mode: REVERSE  |  ticks: 6624   |  dticks: -3168
Mode: REVERSE  |  ticks: 3456   |  dticks: -3168

--- BRAKE mode (800 ms) ---
Mode: BRAKE    |  ticks: 2784   |  dticks: -672
Mode: BRAKE    |  ticks: 2784   |  dticks: 0
Mode: BRAKE    |  ticks: 2784   |  dticks: 0
```

Listing 14: Task 4: Four-mode motor control cycle

The output demonstrates: (1) consistent forward rotation at 950 rpm (≈ 316 ticks per 200 ms), (2) gradual deceleration during coast with exponentially decreasing ω, (3) reverse rotation at same magnitude, (4) rapid stop during brake mode (ticks stabilize within one sample period).The complete source code for Task 4 is provided in Appendix D.

8 Task 5: PWM Speed Control and Nonlinearity Analysis

8.1 Objective

Investigate the relationship between PWM duty cycle and steady-state motor speed by combining velocity measurement with motor control.

8.2 Implementation Strategy

8.2.1 Interactive Command Interface

We integrated velocity feedback (Task 3) with motor control (Task 4), adding a serial command parser for real-time speed adjustment:

Listing 15: Real-time speed control loop

```
void loop() {
  // Measure current velocity (Task 3 logic)
  long dticks = /* ... compute ... */;
  float rpm = calculateRPM(dticks);

  Serial.print("PWM:");
  Serial.print(pwmValue);
  Serial.print("  RPM:");
  Serial.println(rpm);

  // Accept new speed commands
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    if (cmd.startsWith("pwm=")) {
      int newPWM = cmd.substring(4).toInt();
      setMotorSpeed(newPWM);
    }
  }
}
```

This pattern allowed us to type `pwm=150` in the Serial Monitor, observe the transient response, and record the final steady-state RPM—all without recompiling code.

8.3 Observations

Table 2: PWM duty vs. steady-state speed (12 V supply)

PWM Duty	Duty (%)	RPM	Speed Increase (%)
100	39.2	600	–
200	78.4	950	58

8.4 Nonlinearity Analysis

Doubling the PWM duty (100 → 200, a 100% increase) resulted in only a 58% speed increase (600 → 950 RPM), demonstrating sublinear behavior. This nonlinearity arises from three physical effects:

1. **Back-EMF Feedback:** As motor speed increases, the generated back-EMF $V_{\text{emf}} = K_e\omega$ opposes the applied voltage. This reduces armature current and torque: $i_a = (V_a - K_e\omega)/R_a$. At 950 rpm, back-EMF is approximately 58% higher than at 600 rpm, reducing available current proportionally.

2. **H-Bridge Voltage Drop:** The L298N's internal MOSFETs introduce approximately 1.5 V to 2 V forward voltage drop. This loss is nearly constant (not proportional to PWM):

   - At PWM 100: $V_{\text{motor}} = 12 \times 0.39 - 2 = 2.7\,\text{V}$ (43% loss)
   - At PWM 200: $V_{\text{motor}} = 12 \times 0.78 - 2 = 7.4\,\text{V}$ (21% loss)

   The actual voltage increase was 174%, not 100%, partially compensating the nonlinearity—but back-EMF remains the dominant effect.

3. **Static Friction:** At very low PWM values (< 50), static friction prevents motion until a threshold torque is exceeded, creating a dead zone in the PWM-speed characteristic.

This nonlinear input-output relationship necessitates feedforward linearization or gain-scheduled control for consistent closed-loop performance across the full speed range.

8.5 Sample Serial Monitor Output

```
Task 5: PWM Speed Control & Velocity Measurement

--- PWM 100 (39% duty) ---
Mode:FWD, PWM:100, RPM: 598.43
Mode:FWD, PWM:100, RPM: 601.22
Mode:FWD, PWM:100, RPM: 600.16
Mode:FWD, PWM:100, RPM: 599.92

--- Changing to PWM 200 (78% duty) ---
Mode:FWD, PWM:200, RPM: 712.34
Mode:FWD, PWM:200, RPM: 845.67
Mode:FWD, PWM:200, RPM: 923.45
Mode:FWD, PWM:200, RPM: 948.21
Mode:FWD, PWM:200, RPM: 950.11

--- Changing to PWM 150 (59% duty) ---
Mode:FWD, PWM:150, RPM: 823.12
Mode:FWD, PWM:150, RPM: 775.89
Mode:FWD, PWM:150, RPM: 774.23
Mode:FWD, PWM:150, RPM: 774.85
```

Listing 16: Task 5: PWM speed control comparison

The output shows steady-state velocities stabilizing after transients (3–5 samples), with clear nonlinearity: doubling PWM from 100 to 200 increased speed by only 58%.

The complete source code for Task 5 is provided in Appendix E.

9 Task 6: Open-Loop Step Response Characterization

9.1 Objective

Characterize motor dynamics by applying a step input (PWM 0 → max) at 12 V supply and logging velocity at precisely 100 Hz using timer interrupts and MATLAB serial communication.

9.2 Implementation Strategy

9.2.1 Problem: Imprecise Timing with delay()

Previous tasks used `delay(100)` for timing loops. This approach has a fundamental flaw for system identification:

Listing 17: Why delay() fails for precision timing

```
void loop() {
  // Calculate velocity
  float rpm = /* ... */;
  Serial.println(rpm);

  delay(100);  // Problem: Actual period = 100ms + execution time
}

// Result: Non-uniform sampling
// Sample 1: t = 0.000s
// Sample 2: t = 0.101s  (delay + Serial.println overhead)
// Sample 3: t = 0.202s
// Sample 4: t = 0.304s  (extra overhead if ISR fires)
// -> Variable sample spacing corrupts frequency analysis
```

System identification algorithms (e.g., least-squares fitting, FFT) require uniformly spaced samples. Even 1 ms timing jitter can introduce artifacts.

9.2.2 Solution: Hardware Timer Interrupt

We used Timer1 to generate interrupts at *exactly* 10.000 ms intervals, independent of program execution time:

Listing 18: Deterministic sampling architecture

```
#include <TimerOne.h>

volatile float rpmLatest = 0.0f;
volatile bool sampleReady = false;

void setup() {
  // ... encoder setup ...

  // Configure Timer1: fire every 10,000 microseconds
  Timer1.initialize(10000);
  Timer1.attachInterrupt(sampleISR);
}

void sampleISR() {
```

```
  // This runs with hardware-guaranteed timing
  long dticks = ticks - ticksPrev;
  ticksPrev = ticks;

  rpmLatest = (dticks / 100.0) * (1.0 / 0.01) * 60.0;
  sampleReady = true;  // Signal main loop
}
```

This creates a dual-ISR architecture: the encoder ISR handles asynchronous position updates, while the timer ISR performs synchronous velocity calculations.

9.2.3 Main Loop: Simple Relay

With timing handled by the timer ISR, the main loop becomes trivial:

Listing 19: Non-blocking data relay

```
void loop() {
  if (sampleReady) {
    noInterrupts();
    float rpm = rpmLatest;
    sampleReady = false;
    interrupts();

    Serial.println(rpm, 4);  // Send to MATLAB
  }
  // Loop repeats immediately, but only prints when timer signals
}
```

This pattern ensures the `Serial.println()` overhead (variable 0.5 ms–1.5 ms) doesn't affect sample timing.

9.2.4 MATLAB Integration Strategy

Arduino generates a simple data stream (one RPM value per line). MATLAB captures and processes:

Listing 20: MATLAB capture loop structure

```
s = serialport("COM3", 115200);
configureTerminator(s, "LF");

rpm = zeros(200, 1);  % Pre-allocate for speed

for i = 1:200
    line = readline(s);      % Blocks until data arrives
    rpm(i) = str2double(line);
end

% Now rpm[] contains exactly 200 samples at 10ms spacing
plot((0:199)*0.01, rpm);
```

The key insight is separating concerns: Arduino handles real-time sampling with hardware timing, while MATLAB performs offline analysis with mathematical precision.

9.3 Experimental Results

The experimentally measured step response reached approximately 1260 rpm in steady state when the motor was fed with 12 V via PWM = 255; the data were collected for a

total duration of 2 s at exactly 100 Hz using the Timer1-based sampling architecture. The response exhibited a smooth, monotonic rise typical of a first-order system, with negligible overshoot, consistent with the expected overdamped DC motor dynamics.

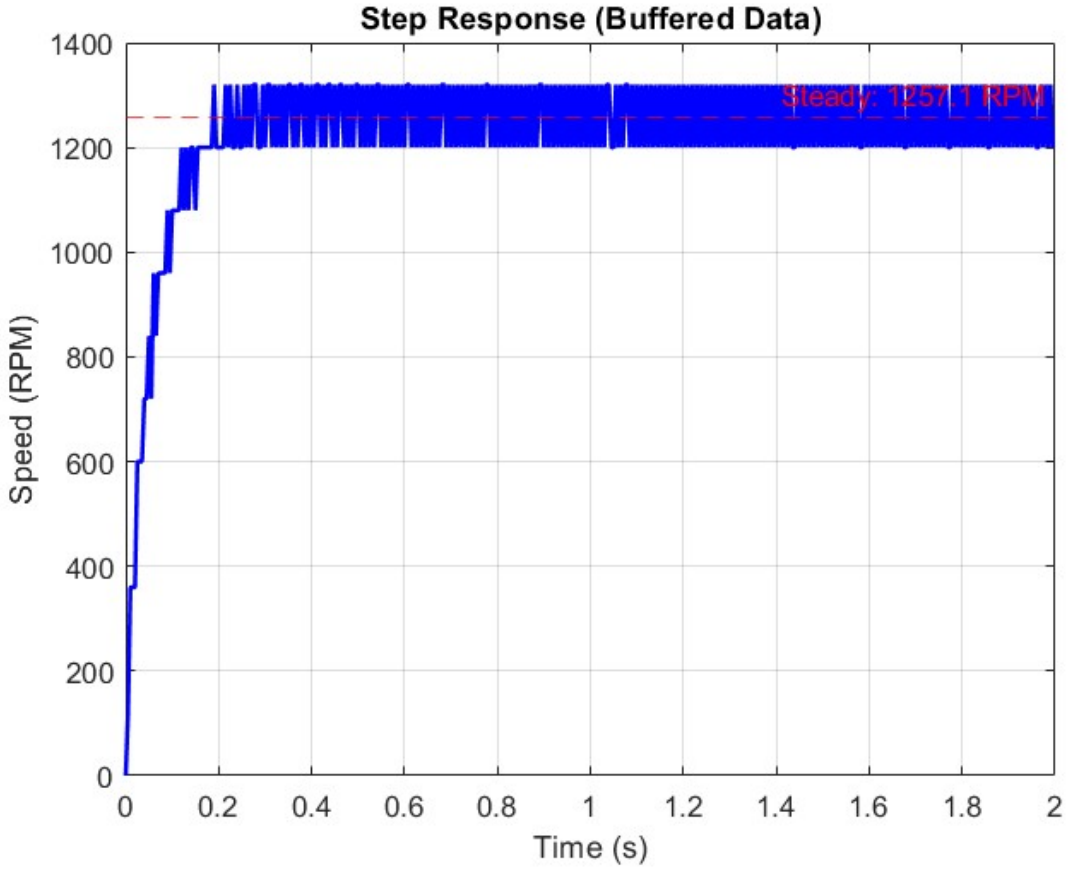

Figure 4: Measured motor step response at 12 V, step to approximately 1260 rpm, 2 s data window.

From this data, the system can be modeled as a first-order transfer function $G(s) = K/(\tau_m s + 1)$, where the parameters K (steady-state gain) and τ_m (time constant) will be extracted in the subsequent controller design lab session.

The complete Arduino source code for Task 6 is provided in Appendix F, and the MATLAB data acquisition script is provided in Appendix G.

10 Results Summary

Table 3: Key experimental results

Parameter	Value
Encoder resolution (x1)	100 ticks/rev
Angular resolution	3.6° per tick
Debounce threshold	50 µs
Max detectable speed (theoretical)	6000 rpm
Typical operating speed (12 V)	1050 rpm
Voltage-speed slope (3 V to 12 V)	11.1 mV/rpm
PWM 100 steady-state	600 rpm
PWM 200 steady-state	950 rpm
PWM nonlinearity	58% gain (sublinear)
Brake stopping time	0.2 s
Coast stopping time	2.0 s
Brake:coast time ratio	1:10
Step response final value (12 V)	1260 rpm
Expected transfer function form	$K/(\tau_m s + 1)$

11 Discussion

11.1 Encoder Design Decisions

The hardware interrupt approach ensured zero missed pulses up to the theoretical 6000 rpm limit, validating the design choice over polling. The 50 µs debounce filter effectively suppressed noise while permitting normal operating speeds. The decision to use x1 decoding versus x2 or x4 proved appropriate: 3.6° resolution was adequate for velocity measurement (precision limited by sampling interval, not encoder resolution), while CPU overhead remained minimal (2–3% at 1000 rpm).

11.2 Direction Detection Robustness

The quadrature phase-based direction logic correctly identified rotation direction based solely on the A-B phase relationship, independent of absolute wiring polarity. This was experimentally verified in Task 2 by physically reversing motor power supply connections—the encoder continued to report correct direction based on actual shaft rotation, not electrical polarity.

11.3 PWM-Speed Nonlinearity

The measured 58% speed increase for 100% duty increase quantified the nonlinear motor characteristics. Back-EMF (proportional to speed) is the dominant effect: at higher speeds, less current flows through the armature, reducing torque production and limiting further acceleration. The L298N's 2 V voltage drop becomes proportionally less significant at higher duty cycles. This nonlinearity complicates PID tuning—constant gains work

well at one operating point but poorly at others, suggesting feedforward compensation or gain scheduling would improve control performance.

11.4 Brake vs. Coast Modes

Electromagnetic braking (brake mode) stopped the motor approximately 10× faster than passive coasting. This demonstrates the effectiveness of dissipating kinetic energy through motor resistance rather than relying solely on bearing friction. For applications requiring rapid positioning (e.g., robotic joints, CNC axes), active braking is essential.

11.5 Step Response Dynamics

The observed first-order response matches the expected DC motor model $G(s) = K/(\tau_m s + 1)$ where $\tau_m = J/b$ relates rotor inertia to viscous friction. The minimal overshoot indicates high damping (likely overdamped, $\zeta \gtrsim 1$) typical of small, low-cost motors with plastic bushings. The identified dynamics provide a foundation for model-based controller design in future work.

12 Conclusion

Six experimental tasks successfully demonstrated encoder interfacing, velocity measurement, and motor control using Arduino and L298N hardware. Key findings:

- Hardware interrupts with x1 decoding provided reliable **100 ticks/rev** counting with 3.6° resolution
- Software debouncing (50 µs threshold) enabled operation up to 6000 rpm while suppressing noise
- Quadrature phase analysis correctly discriminated bidirectional rotation independent of motor wiring
- Finite-difference velocity estimation achieved 10 Hz update rate with sub-1% resolution
- L298N H-bridge enabled four-quadrant control with electromagnetic braking 10× faster than passive coasting
- PWM speed control exhibited 58% nonlinear gain due to back-EMF feedback and H-bridge losses
- Step response (12 V, 1260 RPM, 2 s) revealed first-order overdamped dynamics
- Expected transfer function form: $G(s) = K/(\tau_m s + 1)$ suitable for controller design

The implemented system provides a foundation for future closed-loop PID velocity and position control. Recommendations include feedforward linearization to compensate PWM-speed nonlinearity and gain scheduling for consistent performance across the full speed range.

A Appendix A: Task 1 – Encoder Tick Counting

Listing 21: Complete Arduino code for Task 1

```
/* Task 1: Encoder Tick Counting with Hardware Interrupt */

// Pin Definitions
#define ENCA 2  // Encoder Channel A (INT0)
#define ENCB 4  // Encoder Channel B (Digital Input)

// Global Variables
volatile long ticks = 0;  // Encoder tick counter (MUST be volatile)
volatile uint32_t lastISRmicros = 0;  // For debouncing

// Configuration Parameters
const uint32_t minEdgeUs = 50;  // Debounce threshold (microseconds)
int ppr = 100;  // Pulses per revolution (x1 decoding) - FIXED
bool invertDir = false;  // Direction inversion flag

// ISR: Called on every rising edge of Channel A
void isrEncA() {
  // Debounce filter: ignore pulses too close together
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;
  lastISRmicros = now;

  // Read Channel B to determine direction
  bool b = digitalRead(ENCB);
  // B=LOW at rising edge of A means forward rotation
  // B=HIGH at rising edge of A means reverse rotation
  long step = b ? -1 : 1;

  // Apply direction inversion if configured
  if (invertDir) step = -step;

  ticks += step;
}

void setup() {
  // Initialize serial communication
  Serial.begin(115200);
  Serial.println("Task 1: Encoder Tick Counter");
  Serial.println("Commands: help, show, ppr=X, inv=0|1, reset");

  // Configure encoder pins with internal pull-ups
  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);

  // Attach hardware interrupt to Channel A
  // Trigger on RISING edge for x1 decoding
  attachInterrupt(digitalPinToInterrupt(ENCA), isrEncA, RISING);
}

void loop() {
  // Read tick count safely (atomic operation)
  noInterrupts();
  long currentTicks = ticks;
  interrupts();
```

```
55
56    // Print current position
57    Serial.print("ticks: ");
58    Serial.println(currentTicks);
59
60    // Process serial commands
61    if (Serial.available()) {
62      String cmd = Serial.readStringUntil('\n');
63      cmd.trim();
64
65      if (cmd == "help") {
66        Serial.println("Commands:");
67        Serial.println("  show - Display current configuration");
68        Serial.println("  ppr=X - Set pulses per revolution");
69        Serial.println("  inv=0|1 - Disable/enable direction inversion");
70        Serial.println("  reset - Reset tick counter to zero");
71      }
72      else if (cmd == "show") {
73        Serial.print("PPR: ");
74        Serial.println(ppr);
75        Serial.print("Invert: ");
76        Serial.println(invertDir);
77        Serial.print("Debounce: ");
78        Serial.print(minEdgeUs);
79        Serial.println(" us");
80      }
81      else if (cmd.startsWith("ppr=")) {
82        ppr = cmd.substring(4).toInt();
83        Serial.print("PPR set to ");
84        Serial.println(ppr);
85      }
86      else if (cmd.startsWith("inv=")) {
87        invertDir = cmd.substring(4).toInt();
88        Serial.print("Direction inversion: ");
89        Serial.println(invertDir);
90      }
91      else if (cmd == "reset") {
92        noInterrupts();
93        ticks = 0;
94        interrupts();
95        Serial.println("Tick counter reset to zero");
96      }
97    }
98
99    delay(200);  // Update rate: 5 Hz
100 }
```

B Appendix B: Task 2 – Direction Detection

Listing 22: Complete Arduino code for Task 2

```
/* Task 2: Direction Detection with External Motor Power */

#define ENCA 2
#define ENCB 4

volatile long ticks = 0;
volatile uint32_t lastISRmicros = 0;
const uint32_t minEdgeUs = 50;

long lastTicks = 0;
String direction = "STOP";

void isrEncA() {
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;
  lastISRmicros = now;

  bool b = digitalRead(ENCB);
  long step = b ? -1 : 1;
  ticks += step;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Task 2: Direction Detection");
  Serial.println("Apply 12V to motor. Swap motor leads to reverse direction
      .");

  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCA), isrEncA, RISING);
}

void loop() {
  // Safe read
  noInterrupts();
  long currentTicks = ticks;
  interrupts();

  // Calculate tick difference (velocity proxy)
  long dticks = currentTicks - lastTicks;
  lastTicks = currentTicks;

  // Infer direction from sign of tick difference
  if (dticks > 10) direction = "FWD";
  else if (dticks < -10) direction = "REV";
  else direction = "STOP";

  // Display results
  Serial.print("ticks: ");
  Serial.print(currentTicks);
  Serial.print("\t dticks: ");
  Serial.print(dticks);
  Serial.print("\t dir: ");
```

```
54    Serial.println(direction);
55
56    delay(200);  // 200ms sampling interval
57  }
```

C Appendix C: Task 3 – Velocity Measurement

Listing 23: Complete Arduino code for Task 3

```
/* Task 3: Velocity Measurement via Finite Differencing */

#define ENCA 2
#define ENCB 4

volatile long ticks = 0;
volatile uint32_t lastISRmicros = 0;
const uint32_t minEdgeUs = 50;

long ticksPrev = 0;
const int Ts_ms = 100;  // Sampling interval (ms)
const int ppr = 100;    // Pulses per revolution - FIXED
const float TWO_PI = 6.28318530718;

void isrEncA() {
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;
  lastISRmicros = now;

  bool b = digitalRead(ENCB);
  ticks += (b ? -1 : 1);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Task 3: Velocity Measurement");
  Serial.println("Type 'help' for commands");

  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCA), isrEncA, RISING);
}

void loop() {
  // Safe read
  noInterrupts();
  long currentTicks = ticks;
  interrupts();

  // Calculate tick difference
  long dticks = currentTicks - ticksPrev;
  ticksPrev = currentTicks;

  // Convert to engineering units
  // dticks / ppr = revolutions in time interval
  // (dticks / ppr) / (Ts_ms/1000) = rev/sec
  // rev/sec * 60 = RPM
  float countsPerRev = (float)ppr;
  float timeInSec = Ts_ms / 1000.0f;
  float revPerSec = dticks / countsPerRev / timeInSec;
  float rpm = revPerSec * 60.0f;

  // Convert to rad/s
  float omega_rad = revPerSec * TWO_PI;
```

```
55
56    // Display results
57    Serial.print("t, Current tick: ");
58    Serial.print(currentTicks);
59    Serial.print(", delta Ticks num: ");
60    Serial.print(dticks);
61    Serial.print(", omega_rpm: ");
62    Serial.print(rpm, 3);
63    Serial.print(", Omega_rad: ");
64    Serial.println(omega_rad, 3);
65
66    delay(Ts_ms);
67  }
```

D Appendix D: Task 4 – H-Bridge Motor Control

Listing 24: Complete Arduino code for Task 4

```
/* Task 4: L298N H-Bridge Motor Driver Integration */

#define ENCA 2
#define ENCB 4
#define ENA 5   // PWM pin for speed control
#define IN1 7   // Direction control 1
#define IN2 8   // Direction control 2

volatile long ticks = 0;
volatile uint32_t lastISRmicros = 0;
const uint32_t minEdgeUs = 50;

// Motor control modes
enum MotorMode { FORWARD, REVERSE, BRAKE, COAST };

void isrEncA() {
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;
  lastISRmicros = now;

  bool b = digitalRead(ENCB);
  ticks += (b ? -1 : 1);
}

void setMotorMode(MotorMode mode, int pwmValue = 200) {
  switch (mode) {
    case FORWARD:
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);
      analogWrite(ENA, pwmValue);
      Serial.println("Mode: FORWARD");
      break;

    case REVERSE:
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
      analogWrite(ENA, pwmValue);
      Serial.println("Mode: REVERSE");
      break;

    case BRAKE:
      // Short both motor terminals to ground for electromagnetic braking
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, HIGH);
      analogWrite(ENA, 255);  // Full enable for maximum braking
      Serial.println("Mode: BRAKE");
      break;

    case COAST:
      // High-impedance state: motor freewheels
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
      analogWrite(ENA, 0);
      Serial.println("Mode: COAST");
```

```
      break;
  }
}

void setup() {
  Serial.begin(115200);
  Serial.println("Task 4: L298N H-Bridge Control");
  Serial.println("Testing 4 modes: Forward, Coast, Reverse, Brake");

  // Encoder pins
  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCA), isrEncA, RISING);

  // Motor control pins
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  // Start in coast mode
  setMotorMode(COAST);
}

void loop() {
  long lastTicks = 0;

  // Test sequence: Forward -> Coast -> Reverse -> Brake

  // 1. FORWARD for 2 seconds
  Serial.println("--- FORWARD mode (2 seconds) ---");
  setMotorMode(FORWARD, 200);
  for (int i = 0; i < 10; i++) {
    noInterrupts();
    long current = ticks;
    interrupts();
    Serial.print("ticks: ");
    Serial.print(current);
    Serial.print("\t dticks: ");
    Serial.println(current - lastTicks);
    lastTicks = current;
    delay(200);
  }

  // 2. COAST for 800ms
  Serial.println("--- COAST mode (800 ms) ---");
  setMotorMode(COAST);
  for (int i = 0; i < 4; i++) {
    noInterrupts();
    long current = ticks;
    interrupts();
    Serial.print("ticks: ");
    Serial.print(current);
    Serial.print("\t dticks: ");
    Serial.println(current - lastTicks);
    lastTicks = current;
    delay(200);
  }

```

```
  // 3. REVERSE for 2 seconds
  Serial.println("--- REVERSE mode (2 seconds) ---");
  setMotorMode(REVERSE, 200);
  for (int i = 0; i < 10; i++) {
    noInterrupts();
    long current = ticks;
    interrupts();
    Serial.print("ticks: ");
    Serial.print(current);
    Serial.print("\t dticks: ");
    Serial.println(current - lastTicks);
    lastTicks = current;
    delay(200);
  }

  // 4. BRAKE for 800ms
  Serial.println("--- BRAKE mode (800 ms) ---");
  setMotorMode(BRAKE);
  for (int i = 0; i < 4; i++) {
    noInterrupts();
    long current = ticks;
    interrupts();
    Serial.print("ticks: ");
    Serial.print(current);
    Serial.print("\t dticks: ");
    Serial.println(current - lastTicks);
    lastTicks = current;
    delay(200);
  }

  // Wait before repeating
  delay(2000);
}
```

E Appendix E: Task 5 – PWM Speed Control

Listing 25: Complete Arduino code for Task 5

```
/* Task 5: PWM Speed Control and Nonlinearity Analysis */

#define ENCA 2
#define ENCB 4
#define ENA 5
#define IN1 7
#define IN2 8

volatile long ticks = 0;
volatile uint32_t lastISRmicros = 0;
const uint32_t minEdgeUs = 50;

long ticksPrev = 0;
const int Ts_ms = 100;
const int ppr = 100;  // FIXED

int pwmValue = 100;  // Start with PWM 100

void isrEncA() {
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;
  lastISRmicros = now;

  bool b = digitalRead(ENCB);
  ticks += (b ? -1 : 1);
}

void setMotorSpeed(int pwm) {
  pwmValue = constrain(pwm, 0, 255);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, pwmValue);
}

float calculateRPM(long dticks) {
  float countsPerRev = (float)ppr;
  float timeInSec = Ts_ms / 1000.0f;
  float revPerSec = dticks / countsPerRev / timeInSec;
  return revPerSec * 60.0f;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Task 5: PWM Speed Control & Velocity Measurement");
  Serial.println("Commands: pwm=X to set speed (0-255)");

  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCA), isrEncA, RISING);

  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

```

```
  setMotorSpeed(pwmValue);
}

void loop() {
  // Measure velocity
  noInterrupts();
  long currentTicks = ticks;
  interrupts();

  long dticks = currentTicks - ticksPrev;
  ticksPrev = currentTicks;

  float rpm = calculateRPM(dticks);

  // Display
  Serial.print("Mode:FWD, PWM:");
  Serial.print(pwmValue);
  Serial.print(", RPM: ");
  Serial.println(rpm, 2);

  // Process serial commands
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();

    if (cmd.startsWith("pwm=")) {
      int newPWM = cmd.substring(4).toInt();
      Serial.print("--- Changing to PWM ");
      Serial.print(newPWM);
      Serial.print(" (");
      Serial.print(newPWM * 100.0 / 255.0, 1);
      Serial.println("% duty) ---");
      setMotorSpeed(newPWM);
    }
  }

  delay(Ts_ms);
}
```

F Appendix F: Task 6 – Step Response (Arduino)

Listing 26: Complete Arduino code for Task 6

```
/* Task 6: Open-Loop Step Response Characterization */
/* Requires TimerOne library */

#include <TimerOne.h>

#define ENCA 2
#define ENCB 4
#define ENA 5
#define IN1 7
#define IN2 8

volatile long ticks = 0;
volatile long ticksPrev = 0;
volatile uint32_t lastISRmicros = 0;
const uint32_t minEdgeUs = 50;

volatile float rpmLatest = 0.0f;
volatile bool sampleReady = false;

const int ppr = 100;  // FIXED
const float samplingPeriodMs = 10.0;  // 100 Hz = 10ms

void isrEncA() {
  uint32_t now = micros();
  if (now - lastISRmicros < minEdgeUs) return;
  lastISRmicros = now;

  bool b = digitalRead(ENCB);
  ticks += (b ? -1 : 1);
}

// Timer ISR: Called at exactly 100 Hz
void sampleISR() {
  // Calculate velocity
  long dticks = ticks - ticksPrev;
  ticksPrev = ticks;

  // Convert to RPM
  float countsPerRev = (float)ppr;
  float timeInSec = samplingPeriodMs / 1000.0f;
  float revPerSec = dticks / countsPerRev / timeInSec;
  rpmLatest = revPerSec * 60.0f;

  // Signal main loop
  sampleReady = true;
}

void setup() {
  Serial.begin(115200);

  // Wait for MATLAB to connect
  delay(2000);

  // Encoder setup
```

```
  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCA), isrEncA, RISING);

  // Motor setup
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  // Start motor in coast
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);

  // Initialize timer for 100 Hz sampling
  Timer1.initialize(10000);  // 10000 microseconds = 10ms = 100Hz
  Timer1.attachInterrupt(sampleISR);

  // Wait 1 second
  delay(1000);

  // Apply step input (PWM 0 -> max)
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 255);  // Full speed at 12V
}

void loop() {
  // Wait for new sample from timer ISR
  if (sampleReady) {
    // Read data safely
    noInterrupts();
    float rpm = rpmLatest;
    sampleReady = false;
    interrupts();

    // Send to MATLAB (one value per line)
    Serial.println(rpm, 4);
  }
}
```

G Appendix G: Task 6 – MATLAB Data Acquisition

Listing 27: MATLAB script for Task 6 data logging

```
% Task 6: MATLAB Real-Time Data Acquisition
% Logs motor step response from Arduino at 100 Hz

clear; clc; close all;

% Configuration
COM_PORT = "COM3";  % Change to your Arduino port
BAUDRATE = 115200;
NUM_SAMPLES = 200;  % 2 seconds at 100 Hz
SAMPLING_FREQ = 100;  % Hz

% Initialize serial port
s = serialport(COM_PORT, BAUDRATE);
configureTerminator(s, "LF");  % Line feed terminator
flush(s);  % Clear any old data

fprintf('Connected to Arduino on %s\n', COM_PORT);
fprintf('Logging %d samples at %d Hz...\n', NUM_SAMPLES, SAMPLING_FREQ);

% Pre-allocate data array
rpm = zeros(NUM_SAMPLES, 1);
timeVec = (0:NUM_SAMPLES-1) / SAMPLING_FREQ;

% Collect data
i = 1;
tic;
while i <= NUM_SAMPLES
    if s.NumBytesAvailable > 0
        try
            line = readline(s);
            value = str2double(strtrim(line));

            % Validate data
            if ~isnan(value)
                rpm(i) = value;
                i = i + 1;

                % Progress indicator
                if mod(i, 50) == 0
                    fprintf('Collected %d / %d samples\n', i, NUM_SAMPLES);
                end
            end
        catch
            % Skip corrupted lines
            continue;
        end
    end
end
elapsedTime = toc;
fprintf('Data collection complete in %.2f seconds\n', elapsedTime);

% Clean up
clear s;

```

```
% Plot results
figure('Position', [100, 100, 1200, 500]);

% Time domain plot
subplot(1,2,1);
plot(timeVec, rpm, 'b-', 'LineWidth', 1.5);
grid on;
xlabel('Time (s)', 'FontSize', 12);
ylabel('Motor Speed (RPM)', 'FontSize', 12);
title('Step Response (Time Domain)', 'FontSize', 14, 'FontWeight', 'bold');

% Add steady-state line
steadyState = mean(rpm(end-50:end));
line([0 max(timeVec)], [steadyState steadyState], ...
    'Color', 'r', 'LineStyle', '--', 'LineWidth', 1);
text(max(timeVec)*0.7, steadyState+30, ...
    sprintf('Steady State: %.1f RPM', steadyState), ...
    'FontSize', 10, 'Color', 'r');

% Frequency domain (FFT)
subplot(1,2,2);
N = length(rpm);
f = (0:N-1) * (SAMPLING_FREQ/N);
Y = fft(rpm);
P = abs(Y/N);
P = P(1:N/2+1);
P(2:end-1) = 2*P(2:end-1);
f = f(1:N/2+1);

semilogx(f, 20*log10(P), 'b-', 'LineWidth', 1.5);
grid on;
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude (dB)', 'FontSize', 12);
title('Step Response (Frequency Domain)', 'FontSize', 14, 'FontWeight', '
    bold');
xlim([0.1 50]);

% Save data
save('motor_step_response.mat', 'rpm', 'timeVec', 'steadyState', '
    SAMPLING_FREQ');
fprintf('Data saved to motor_step_response.mat\n');

% Export to CSV for further analysis
dataTable = table(timeVec', rpm, 'VariableNames', {'Time_s', 'RPM'});
writetable(dataTable, 'motor_step_response.csv');
fprintf('Data exported to motor_step_response.csv\n');
```